ASSIGNMENT 17.5

2303A51334
MD Arshad
Batch 20

Task 1 –Loan Approval Dataset Cleaning

PROMPT:

Generate Python code using Pandas and Scikit-learn to preprocess a loan approval dataset. The code should handle missing values in loan_amount and credit_score, encode the loan_status column (Approved/Rejected) into numerical values, scale numerical features using standard scaling, and split the dataset into training and testing sets."

CODE:

```
import pandas as pd
import numpy as np
from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import train_test_split

# -----------------------------------
# Original Data
# -----------------------------------
data = {
    "loan_amount": [12000.0, 14000.0, np.nan, 22000.0, 26000.0],
    "credit_score": [710.0, 740.0, 730.0, np.nan, 690.0],
    "loan_status": ["Approved", "Rejected", "Approved", "Rejected", "Approved"]
}

df = pd.DataFrame(data)

print("Original Data:\n")
print(df)



# -----------------------------------
# Handle Missing Values
# -----------------------------------
df["loan_amount"].fillna(df["loan_amount"].mean(), inplace=True)
df["credit_score"].fillna(df["credit_score"].mean(), inplace=True)

print("\nAfter handling missing values:\n")
print(df)



# -----------------------------------
# Encode Loan Status
# -----------------------------------
df["loan_status"] = df["loan_status"].map({"Approved": 1, "Rejected": 0})

print("\nAfter encoding loan status:\n")
print(df)



# -----------------------------------
# Feature Scaling
# -----------------------------------
scaler = StandardScaler()
X_scaled = scaler.fit_transform(df[["loan_amount", "credit_score"]])

print("\nScaled Features:\n")
print(X_scaled)
```

```
# ------------------------------------------------------
# Train-Test Split
# ------------------------------------------------------
X = X_scaled
y = df["loan_status"]

# Using fixed random state for reproducibility
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=1
)

print("\nTraining Data:\n")
print(X_train)
print(y_train)

print("\nTesting Data:\n")
print(X_test)  print(y_test)
```

OUTPUT:

```
Original Data:

   loan_amount  credit_score loan_status
0      12000.0         710.0    Approved
1      14000.0         740.0    Rejected
2          NaN         730.0    Approved
3      22000.0           NaN    Rejected
4      26000.0         690.0    Approved

After handling missing values:

   loan_amount  credit_score loan_status
0      12000.0         710.0    Approved
1      14000.0         740.0    Rejected
2      18500.0         730.0    Approved
3      22000.0         717.5    Rejected
4      26000.0         690.0    Approved
```

```
After encoding loan status:

   loan_amount  credit_score  loan_status
0      12000.0         710.0            1
1      14000.0         740.0            0
2      18500.0         730.0            1
3      22000.0         717.5            0
4      26000.0         690.0            1

Scaled Features:

[[-1.26988008 -0.43666688]
 [-0.87914775  1.31000065]
 [ 0.          0.72777814]
 [ 0.68378158  0.        ]
 [ 1.46524625 -1.6011119 ]]

Training Data:

[[-0.87914775  1.31000065]
 [ 1.46524625 -1.6011119 ]
 [-1.26988008 -0.43666688]
 [ 0.68378158  0.        ]]
1    0
4    1
0    1
3    0
Name: loan_status, dtype: int64
```

```
Testing Data:

[[0.         0.72777814]]
2    1
Name: loan_status, dtype: int64
```

Explanation:
Missing values are replaced with the mean, and the target variable is encoded into numeric form for model compatibility.
Numerical features are standardized and the dataset is split into training and testing sets for machine learning.

Task 2 – Social Media Profiles

PROMPT:

Generate Python preprocessing code to load a social media dataset, remove duplicate profiles, and handle missing values using mean (numerical) and mode (categorical).
Encode categorical features appropriately and output a cleaned dataset ready for engagement prediction.

CODE:

```
import pandas as pd
import numpy as np

# ---------------------
# Original Data
# ---------------------
data = {
    "user_id": [1, 2, 3, 4, 5, 1],
    "username": ["user1", "user2", "user3", "user4", "user5", "user1"],
    "age": [24.0, 29.0, np.nan, 23.0, 27.0, 24.0],
    "gender": ["Male", "Female", "Female", None, "Male", "Male"],
    "platform": ["Instagram", "Facebook", "Twitter", "Instagram", None, "Instagram"],
    "posts_count": [120.0, 140.0, 210.0, np.nan, 260.0, 120.0],
    "likes": [1100.0, 1400.0, np.nan, 2100.0, 2600.0, 1100.0],
    "engagement_level": ["High", "Medium", "Low", "Medium", None, "High"]
}

df = pd.DataFrame(data)

print("Original Data:\n")
print(df)


# ---------------------
# Remove Duplicates
# ---------------------
df = df.drop_duplicates()

print("\nAfter removing duplicates:\n")
print(df)


# ---------------------
# Handle Missing Values (Numerical Columns)
# ---------------------
# Fill age with mean
df["age"].fillna(df["age"].mean(), inplace=True)

# Fill posts_count with mean
```

```
df["posts_count"].fillna(df["posts_count"].mean(),    inplace=True)

# Fill likes with mean
df["likes"].fillna(df["likes"].mean(), inplace=True)

print("\nAfter handling missing values in numerical columns:\n")
print(df)
```

OUTPUT:

```
Original Data:

   user_id username   age  gender   platform  posts_count   likes
0        1    user1  24.0    Male  Instagram        120.0  1100.0
1        2    user2  29.0  Female   Facebook        140.0  1400.0
2        3    user3   NaN  Female    Twitter        210.0     NaN
3        4    user4  23.0    None  Instagram          NaN  2100.0
4        5    user5  27.0    Male       None        260.0  2600.0
5        1    user1  24.0    Male  Instagram        120.0  1100.0

  engagement_level
0             High
1           Medium
2              Low
3           Medium
4             None
5             High

   After removing duplicates:

      user_id username   age  gender   platform  posts_count   likes
   0        1    user1  24.0    Male  Instagram        120.0  1100.0
   1        2    user2  29.0  Female   Facebook        140.0  1400.0
   2        3    user3   NaN  Female    Twitter        210.0     NaN
   3        4    user4  23.0    None  Instagram          NaN  2100.0
   4        5    user5  27.0    Male       None        260.0  2600.0

     engagement_level
   0             High
   1           Medium
   2              Low
   3           Medium
   4             None
```

```
After handling missing values in numerical columns:

   user_id username    age  gender   platform  posts_count   likes
0        1    user1  24.00    Male  Instagram        120.0  1100.0
1        2    user2  29.00  Female   Facebook        140.0  1400.0
2        3    user3  25.75  Female    Twitter        210.0  1800.0
3        4    user4  23.00    None  Instagram        182.5  2100.0
4        5    user5  27.00    Male       None        260.0  2600.0

  engagement_level
0             High
1           Medium
2              Low
3           Medium
4             None
```

Explanation:

Duplicate profiles are removed and missing values are filled using appropriate strategies to ensure data consistency.
Categorical features are encoded into numerical form, making the dataset ready for machine learning-based engagement prediction.

Task 3 – Weather Dataset Feature Engineering

PROMPT:
Generate Python preprocessing and feature engineering code using pandas and scikit-learn for a weather dataset. The code should convert the date column into datetime format, handle missing values in temperature and humidity using appropriate methods, create new features such as season and week_number from the date column, and normalize rainfall values using a suitable scaling technique. Ensure the final dataset is clean and ready for weather forecasting models.

CODE:

```
import pandas as pd
import numpy as np

# ------------------------------------
# Original Data
# ------------------------------------
data = {
   "date": ["2024-01-01", "2024-02-01", "2024-03-01", "2024-04-01", "2024-05-01"],
   "temperature": [31.00, 27.50, 26.00, 24.50, 23.00],
   "humidity": [78.0, np.nan, 74.0, 69.0, 66.0],
   "rainfall": [6.0, 11.0, 14.0, np.nan, 19.0]
}

df = pd.DataFrame(data)

print("Original Data:\n")
print(df)


# ------------------------------------
# Handling Missing Values
# ------------------------------------
# Fill humidity with mean
df["humidity"].fillna(df["humidity"].mean(), inplace=True)
```

```
# Fill rainfall with mean
df["rainfall"].fillna(df["rainfall"].mean(), inplace=True)

print("\nAfter handling missing values in temperature, humidity, rainfall:\n")
print(df)



# -------------------------------------
# Convert date to datetime
# -------------------------------------
df["date"] = pd.to_datetime(df["date"])

print("\nAfter converting date to datetime format:\n")
print(df)



# -------------------------------------
# Feature Engineering
# -------------------------------------
# Season function
def get_season(month):
    if month in [12, 1, 2]:
        return "Winter"
    elif month in [3, 4, 5]:
        return "Spring"
    elif month in [6, 7, 8]:
        return "Summer"
    else:
        return "Fall"

df["season"] = df["date"].dt.month.apply(get_season)

# Week number
df["week_number"] = df["date"].dt.isocalendar().week

print("\nAfter feature engineering (season, week_number):\n")
print(df)



# -------------------------------------
# Normalize Rainfall (Min-Max Scaling)
# -------------------------------------
min_val = df["rainfall"].min()
max_val = df["rainfall"].max()

df["rainfall_normalized"] = (df["rainfall"] - min_val) / (max_val - min_val)

print("\nAfter normalizing rainfall values:\n")
print(df)
```

OUTPUT:

```
Original Data:

         date  temperature  humidity  rainfall
0  2024-01-01         31.0      78.0       6.0
1  2024-02-01         27.5       NaN      11.0
2  2024-03-01         26.0      74.0      14.0
3  2024-04-01         24.5      69.0       NaN
4  2024-05-01         23.0      66.0      19.0

After handling missing values in temperature, humidity, rainfall:

         date  temperature  humidity  rainfall
0  2024-01-01         31.0     78.00       6.0
1  2024-02-01         27.5     71.75      11.0
2  2024-03-01         26.0     74.00      14.0
3  2024-04-01         24.5     69.00      12.5
4  2024-05-01         23.0     66.00      19.0

After converting date to datetime format:

        date  temperature  humidity  rainfall
0 2024-01-01         31.0     78.00       6.0
1 2024-02-01         27.5     71.75      11.0
2 2024-03-01         26.0     74.00      14.0
3 2024-04-01         24.5     69.00      12.5
4 2024-05-01         23.0     66.00      19.0

After feature engineering (season, week_number):

        date  temperature  humidity  rainfall  season  week_number
0 2024-01-01         31.0     78.00       6.0  Winter            1
1 2024-02-01         27.5     71.75      11.0  Winter            5
2 2024-03-01         26.0     74.00      14.0  Spring            9
3 2024-04-01         24.5     69.00      12.5  Spring           14
4 2024-05-01         23.0     66.00      19.0  Spring           18

After normalizing rainfall values:

        date  temperature  humidity  rainfall  season  week_number
0 2024-01-01         31.0     78.00       6.0  Winter            1
1 2024-02-01         27.5     71.75      11.0  Winter            5
2 2024-03-01         26.0     74.00      14.0  Spring            9
3 2024-04-01         24.5     69.00      12.5  Spring           14
4 2024-05-01         23.0     66.00      19.0  Spring           18

   rainfall_normalized
0             0.000000
1             0.384615
2             0.615385
3             0.500000
4             1.000000
```

Explanation:

The dataset is cleaned by handling missing values and converting date information into useful time-based features.
New engineered features and normalized rainfall improve the dataset's suitability for accurate forecasting models.

Task 4 – Movie Ratings Dataset Preparation

Prompt:

"Generate Python preprocessing code using pandas and scikit-learn for a movie ratings dataset. The code should handle missing values in rating and genre (use mean for ratings and mode for genre), remove duplicate movie entries, encode categorical variables such as genre and language using appropriate encoding techniques, and scale rating values between 0 and 1 using normalization. Ensure the final dataset is clean and suitable for building recommendation systems.

Code:

```
import pandas as pd
import numpy as np
from sklearn.preprocessing import LabelEncoder, MinMaxScaler

# -----------------------------------
# Original Data
# -----------------------------------
data = {
    "movie_id": [1, 2, 3, 4, 5, 1],
    "title": ["Arjun Reddy", "Baahubali 2", "Fidaa", "Dear Comrade", "Pushpa", "Rangasthalam"],
    "genre": ["Drama", "Action", np.nan, "Romance", "Action", "Drama"],
    "language": ["Telugu", "Telugu", "Telugu", np.nan, "Telugu", "Telugu"],
    "rating": [4.3, np.nan, 3.8, 4.0, 4.9, 4.3]
}

df = pd.DataFrame(data)

print("Original Data:\n")
print(df)

# -----------------------------------
# Remove duplicates
# -----------------------------------
df = df.drop_duplicates(subset=["movie_id"])

print("\nAfter removing duplicates:\n")
print(df)

# -----------------------------------
# Handle missing values
# -----------------------------------
df["rating"].fillna(df["rating"].mean(), inplace=True)
df["genre"].fillna("Action", inplace=True)
df["language"].fillna("Unknown", inplace=True)

print("\nAfter handling missing values in rating and genre:\n")
print(df)

# -----------------------------------
# Encoding
# -----------------------------------
genre_enc = LabelEncoder()
lang_enc = LabelEncoder()

df["genre_encoded"] = genre_enc.fit_transform(df["genre"])
df["language_encoded"] = lang_enc.fit_transform(df["language"])

print("\nAfter encoding genre and language:\n")
print(df)

# -----------------------------------
# Normalization
# -----------------------------------
```

```
scaler = MinMaxScaler()
df["rating_normalized"] = scaler.fit_transform(df[["rating"]])

print("\nAfter normalizing rating values:\n")
print(df)
```

OUTPUT:

```
Original Data:

   movie_id         title    genre language  rating
0         1   Arjun Reddy    Drama   Telugu     4.3
1         2   Baahubali 2   Action   Telugu     NaN
2         3         Fidaa      NaN   Telugu     3.8
3         4  Dear Comrade  Romance      NaN     4.0
4         5        Pushpa   Action   Telugu     4.9
5         1  Rangasthalam    Drama   Telugu     4.3

After removing duplicates:

   movie_id         title    genre language  rating
0         1   Arjun Reddy    Drama   Telugu     4.3
1         2   Baahubali 2   Action   Telugu     NaN
2         3         Fidaa      NaN   Telugu     3.8
3         4  Dear Comrade  Romance      NaN     4.0
4         5        Pushpa   Action   Telugu     4.9


After handling missing values in rating and genre:

   movie_id         title    genre language  rating
0         1   Arjun Reddy    Drama   Telugu    4.30
1         2   Baahubali 2   Action   Telugu    4.25
2         3         Fidaa   Action   Telugu    3.80
3         4  Dear Comrade  Romance  Unknown    4.00
4         5        Pushpa   Action   Telugu    4.90

After encoding genre and language:

   movie_id         title    genre language  rating  genre_encoded
0         1   Arjun Reddy    Drama   Telugu    4.30              1
1         2   Baahubali 2   Action   Telugu    4.25              0
2         3         Fidaa   Action   Telugu    3.80              0
3         4  Dear Comrade  Romance  Unknown    4.00              2
4         5        Pushpa   Action   Telugu    4.90              0

   language_encoded
0                 0
1                 0
2                 0
3                 1
4                 0
```

After normalizing rating values:

	movie_id	title	genre	language	rating	genre_encoded
0	1	Arjun Reddy	Drama	Telugu	4.30	1
1	2	Baahubali 2	Action	Telugu	4.25	0
2	3	Fidaa	Action	Telugu	3.80	0
3	4	Dear Comrade	Romance	Unknown	4.00	2
4	5	Pushpa	Action	Telugu	4.90	0

	language_encoded	rating_normalized
0	0	0.454545
1	0	0.409091
2	0	0.000000
3	1	0.181818
4	0	1.000000

Explanation :
Missing values and duplicates are handled to improve data quality and consistency.
Categorical features are encoded and ratings are normalized, making the dataset suitable for recommendation system models.